

ATELIER 1 — Séance 2

Démarrage du projet Spring Boot + Maven et première entité JPA

Support technique: Java 17+ / Maven / IntelliJ IDEA / Spring Boot / Spring Data JPA / Lombok

Objectifs pédagogiques

- Créer un projet Spring Boot / Maven avec IntelliJ IDEA, prêt pour la persistance JPA.
- Comprendre l'arborescence générée et le rôle du fichier pom.xml.
- Configurer la connexion à une base de données MySQL et les propriétés Hibernate.
- Créer une première entité JPA propre (Vehicule), en appliquant Lombok et les bonnes pratiques Clean Code.
- Vérifier la génération automatique du schéma en base (DDL) au démarrage de l'application.

Contexte

Le dépôt Git « AutoLoc » et l'environnement de travail ont été préparés à l'Atelier 0. Il s'agit maintenant de faire naître le squelette technique du projet « autoloc-api » : un vrai projet Spring Boot connecté à une base de données, doté de sa toute première entité de domaine, Vehicule. Ce socle servira de fondation à tous les ateliers suivants.

Prérequis (Atelier 0)

- JDK 17+ installé et configuré dans IntelliJ IDEA.
- IntelliJ IDEA (Community ou Ultimate) installé.
- Un SGBD MySQL installé et démarré (ou accessible), avec un client (MySQL Workbench, DBeaver...).
- Postman (ou équivalent) installé.
- Dépôt Git « AutoLoc » créé, avec accès en écriture.

2. Étapes détaillées — Création du projet Spring Boot + Maven avec IntelliJ IDEA

Étape 1 — Créer le projet via Spring Initializr intégré à IntelliJ

1. Ouvrir IntelliJ IDEA → écran d'accueil → cliquer sur « New Project ».
2. Dans le panneau de gauche, sélectionner « Spring Boot » (Spring Initializr est intégré nativement à IntelliJ Ultimate ; à défaut, utiliser start.spring.io puis « Open » le projet généré dans IntelliJ Community).

3. Renseigner les champs du générateur :

- Name : autoloc-api
- Location : dossier local du dépôt Git « AutoLoc »
- Language : Java
- Type : Maven
- Group : tn.esprit.autoloc
- Artifact : autoloc-api
- Package name : tn.esprit.autoloc
- JDK : 17 (ou version supérieure installée)
- Java : 17
- Packaging : Jar

4. Cliquer sur « Next » pour passer à l'écran de sélection des dépendances (« Dependencies »).

5. Sélectionner les dépendances suivantes (rechercher chaque nom dans la barre de recherche) :

- Spring Web
- Spring Data JPA
- MySQL Driver
- Lombok
- Validation
- Spring Boot DevTools

6. Cliquer sur « Create » (ou « Finish »). IntelliJ génère le projet et lance automatiquement l'import Maven (téléchargement des dépendances) : attendre la fin du téléchargement (barre de progression en bas à droite).

Astuce

Si le projet a été généré sur start.spring.io, télécharger le .zip, le décompresser dans le dossier du dépôt « AutoLoc », puis dans IntelliJ : File → Open... → sélectionner le dossier « autoloc-api » → « Open as Project ».

Étape 2 — Vérifier la structure du projet généré

Une fois l'import Maven terminé, l'arborescence attendue dans le panneau « Project » est la suivante:

```

autoloc-api
├── src
│   ├── main
│   │   ├── java/tn/esprit/autoloc
│   │   │   └── AutolocApiApplication.java
│   │   └── resources
│   │       ├── application.properties
│   │       ├── static/
│   │       └── templates/
│   └── test
│       └── java/tn/esprit/autoloc
└── pom.xml
  
```

Ouvrir le fichier pom.xml et vérifier que les dépendances sélectionnées y figurent bien (spring-boot-starter-web, spring-boot-starter-data-jpa, mysql-connector-j, lombok, spring-boot-starter-validation, spring-boot-devtools).

Étape 3 — Activer l'annotation processing pour Lombok

7. Aller dans File → Settings (ou Preferences sur Mac) → Build, Execution, Deployment → Compiler → Annotation Processors.
8. Cocher « Enable annotation processing ».
9. Vérifier que le plugin Lombok est installé (File → Settings → Plugins → rechercher « Lombok » ; l'installer et redémarrer IntelliJ si besoin).

IMPORTANT

Sans l'activation de l'annotation processing, les getters/setters générés par Lombok ne seront pas reconnus par IntelliJ : le code semblera en erreur alors qu'il compile correctement en ligne de commande.

Étape 4 — Configurer application.properties

Ouvrir src/main/resources/application.properties et renseigner la configuration suivante :

```
spring.application.name=autoloc-api

# Connexion à la base de données
spring.datasource.url=jdbc:mysql://localhost:3306/autoloc_db?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=votre_mot_de_passe

# Hibernate / JPA
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Logs
logging.level.org.hibernate.SQL=DEBUG
logging.level.tn.esprit.autoloc=INFO
```

Indication technique

Le paramètre **createDatabaseIfNotExist=true**, ajouté à l'URL JDBC, demande au pilote MySQL de créer automatiquement la base autoloc_db si elle n'existe pas encore : il n'est donc plus nécessaire de la créer manuellement au préalable.

spring.jpa.hibernate.ddl-auto=update permet ensuite à Hibernate de créer/mettre à jour automatiquement les tables à partir des entités. Ces réglages sont adaptés au développement ; ils ne doivent jamais être utilisés tels quels en production.

Les stratégies spring.jpa.hibernate.ddl-auto

Valeur	Comportement	Quand l'utiliser
none	Hibernate ne touche pas au schéma.	Production, quand le schéma est géré par un outil de migration (Liquibase, Flyway) ou manuellement.

Valeur	Comportement	Quand l'utiliser
validate	Hibernate compare les entités au schéma existant et lève une erreur au démarrage si elles ne correspondent pas, sans rien modifier.	Environnements sensibles (pré-production, production) : garde-fou pour détecter un décalage entre le code et la base.
update	Hibernate ajoute les tables/colonnes manquantes pour faire correspondre le schéma aux entités, sans supprimer ni renommer les colonnes existantes (une colonne renommée dans le code sera dupliquée, pas migrée).	Développement au quotidien (choix fait pour AutoLoc) : pratique pour itérer rapidement sur le modèle.
create	Hibernate supprime les tables existantes puis les recrée à chaque démarrage, à partir des entités. Toutes les données sont perdues.	Tests ponctuels où l'on veut repartir d'un schéma propre, jamais avec des données à conserver.
create-drop	Comme create, mais supprime en plus le schéma à l'arrêt propre de l'application (utile avec une base embarquée H2).	Tests automatisés (@DataJpaTest, intégration) : base jetable créée puis détruite à chaque exécution.

Étape 5 — Créer l'arborescence des packages du projet

Sous `tn.esprit.autoloc`, créer les packages suivants (clic droit sur le package → New → Package), qui seront utilisés dès cet atelier ou lors des suivants :

```
tn.esprit.autoloc
├── domain          (entités JPA, énumérations)
├── repository      (interfaces Spring Data JPA)
├── service         (couche métier – Atelier 4)
├── web.controller (contrôleurs REST – Atelier 5)
└── web.dto         (DTO – Atelier 6)
```

Étape 6 — Créer la première entité JPA : Vehicule

Dans le package `domain`, créer l'énumération `StatutVehicule` :

```
package tn.esprit.autoloc.domain;

public enum StatutVehicule {
    DISPONIBLE, LOUE, MAINTENANCE
}
```

Puis l'énumération `CategorieVehicule` :

```
package tn.esprit.autoloc.domain;

public enum CategorieVehicule {
    CITADINE, BERLINE, SUV, UTILITAIRE
}
```

Enfin, créer la classe `Vehicule` (sans association pour l'instant — les associations seront ajoutées à l'Atelier 2) :

```

package tn.esprit.autoloc.domain;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.math.BigDecimal;

@Entity
@Table(name = "vehicule")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Vehicule {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idVehicule;

    @Column(nullable = false, unique = true, length = 20)
    private String immatriculation;

    @Column(nullable = false, length = 50)
    private String marque;

    @Column(nullable = false, length = 50)
    private String modele;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private CategorieVehicule categorie;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal tarifJournalier;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private StatutVehicule statut;
}

```

Indication technique

Préférer @Getter/@Setter ciblés à @Data sur les entités qui auront des associations bidirectionnelles (Atelier 2), pour éviter les boucles infinies de equals/hashCode/toString.

Les stratégies @GeneratedValue des identifiants

Dans une entité JPA, @Id désigne l'attribut qui sert de clé primaire. L'annotation @GeneratedValue, placée juste à côté, indique à Hibernate comment cette valeur doit être générée automatiquement plutôt que fournie par le code applicatif. Le paramètre strategy accepte une valeur de l'énumération GenerationType, qui compte quatre stratégies.

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idVehicule;
```

Les quatre stratégies de GenerationType

Stratégie	Principe	Avantages / limites
IDENTITY	Délègue la génération à une colonne auto-incrémentée gérée par le SGBD (AUTO_INCREMENT en MySQL). Hibernate exécute l'INSERT puis récupère l'identifiant généré.	Simple et nativement supportée par MySQL. Limite : désactive le batching des INSERT chez Hibernate (chaque insertion doit être exécutée immédiatement pour connaître l'id), ce qui peut coûter en performance sur de gros volumes.
SEQUENCE	S'appuie sur une séquence gérée par le SGBD (objet dédié qui distribue des valeurs uniques), interrogée avant l'INSERT.	Compatible avec le batching, généralement la stratégie la plus performante. Non disponible sur MySQL avant la version 8.0 (les séquences n'existaient pas) ; couramment utilisée avec PostgreSQL, Oracle, H2.
TABLE	Simule une séquence à l'aide d'une table dédiée qui stocke le dernier identifiant attribué pour chaque entité.	Fonctionne sur tous les SGBD, y compris ceux sans séquence native. Coûteuse en performance (verrous, requêtes supplémentaires) : à éviter sauf portabilité stricte exigée.
AUTO	Laisse le fournisseur JPA (Hibernate) choisir la stratégie la plus adaptée au dialecte du SGBD configuré.	Pratique pour démarrer ou pour rester portable entre SGBD, mais le comportement réel dépend de la version d'Hibernate et du dialecte : moins prévisible dans un contexte pédagogique où l'on veut illustrer un choix explicite.

Exemples de déclaration

IDENTITY (choix retenu pour AutoLoc avec MySQL)

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long idVehicule;
```

SEQUENCE (nécessite de déclarer la séquence)

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "vehicule_seq")
@SequenceGenerator(name = "vehicule_seq", sequenceName = "vehicule_seq", allocationSize = 1)
private Long idVehicule;
```

TABLE (portable mais coûteuse)

```
@Id
@GeneratedValue(strategy = GenerationType.TABLE, generator = "vehicule_gen")
@TableGenerator(name = "vehicule_gen", table = "id_generator", pkColumnName = "vehicule_id")
private Long idVehicule;
```

Étape 7 — Lancer l'application et vérifier la génération du schéma

10. Ouvrir AutolocApiApplication.java et lancer l'application (clic droit → Run, ou bouton ► en haut à droite d'IntelliJ).
11. Observer la console : la ligne « Started AutolocApiApplication in ... seconds » confirme le démarrage réussi.
12. Repérer dans les logs les requêtes DDL générées par Hibernate (create table vehicule ...).
13. Se connecter à la base autoloc_db avec le client MySQL et vérifier que la table vehicule a bien été créée, avec les bonnes colonnes et types.
14. Committer et pousser le travail sur le dépôt Git avec un message clair (ex. « Atelier 1 : init projet Spring Boot + entité Vehicule »).

3. Livrables attendus

- Projet Spring Boot exécutable, connecté à la base de données MySQL.
- Entité Vehicule propre (Lombok ciblé, sans getters/setters manuels).
- Fichier **application.properties** correctement configuré et versionné (hors mot de passe en clair si possible — variable d'environnement recommandée).
- Logs applicatifs lisibles (niveau INFO au démarrage, DEBUG pour le SQL).
- Premier commit poussé sur le dépôt Git « AutoLoc ».

4. Critères d'évaluation

- Respect des conventions de nommage Java / Spring.
- Utilisation correcte et minimale de Lombok (pas d'abus de @Data sur des entités qui auront des relations).
- Absence de code mort ou de configuration inutile (Clean Code).
- Schéma généré en base conforme à l'entité définie.
- Qualité du message de commit Git.

5. Travail à la maison — Finaliser les entités restantes

Afin d'arriver préparé à l'Atelier 2 (modélisation complète des associations, cascade et fetch — Séance 3), chaque étudiant doit, avant la prochaine séance, créer — sur le même modèle que Vehicule — les entités JPA restantes du modèle AutoLoc, sans association pour l'instant (les relations seront ajoutées collectivement en Atelier 2).

Entités à créer dans le package domain

Entité	Attributs principaux (sans association)
Agence	idAgence, nom, ville, adresse, telephone
Client	idClient, nom, prenom, email, telephone, numPermis, dateInscription

Entité	Attributs principaux (sans association)
Employe	idEmploye, nom, prenom, role (enum RoleEmploye : AGENT, MANAGER)
Equipement	idEquipement, libelle
Reservation	idReservation, dateDebut, dateFin, statut (enum StatutReservation : EN_ATTENTE, CONFIRMEE, ANNULEE, TERMINEE)
Contrat	idContrat, dateSignature, montantTotal, valide
Paieement	idPaieement, montant, datePaieement, modePaieement (enum ModePaieement : CARTE, ESPECES, VIREMENT)
Maintenance	idMaintenance, dateDebut, dateFin, description

Consignes

- Créer chaque entité avec `@Entity`, `@Id`, `@GeneratedValue(strategy = GenerationType.IDENTITY)` et les annotations Lombok ciblées (`@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`).
- Créer les énumérations demandées (RoleEmploye, StatutReservation, ModePaieement) dans le package domain, sur le modèle de StatutVehicule.
- Ne pas ajouter les associations : elles seront étudiées et justifiées lors de l'Atelier 2 (Séance 3).
- Relancer l'application et vérifier dans les logs que toutes les tables sont bien créées en base, avec les bons types de colonnes.
- Commiter le travail sur le dépôt Git avec un message dédié (ex. « Prépa Atelier 2 : entités restantes sans associations »).

À rendre / vérifier avant la Séance 3

Neuf entités au total (Vehicule + les 8 ci-dessus), toutes compilables, toutes générant leur table en base, et un dépôt Git à jour. Aucune association n'est attendue à ce stade : elle sera construite pas à pas en Atelier 2.

6. Pour aller plus loin (facultatif)

- Ajouter un **CommandLineRunner** qui insère 2 ou 3 véhicules de démonstration au démarrage
- Configurer un profil « dev » (application-dev.properties) distinct de la configuration par défaut

Passerelle vers l'Atelier 2

Cette première entité Vehicule sera enrichie dès la séance suivante avec l'ensemble des associations du modèle AutoLoc (Agence, Client, Reservation, Contrat, Equipement...) et les stratégies de cascade / fetch associées.